Übungsblatt 4: Aktorik & Sicherheit

Mikrocomputertechnik 3 · Zeitdiagramme, PWM-Rechnungen und überwachte Abläufe

Ziel und Arbeitsweise

Sie berechnen Hardware-PWM, lesen Zeitdiagramme und beurteilen, ob ein Watchdog fehlenden Anwendungsfortschritt erkennen kann. Alle sechs Pflichtaufgaben sind als Papieraufgaben ohne Platine oder Simulator lösbar.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht sind 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 min), Aufgaben 2–3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Besprechungen gehören zu den Vorlesungsphasen.

Gemeinsames Modell: Der PWM-Timer erhält $f_{\text{tim}} = 100$ MHz. PSC, Prescaler oder Vorteiler, teilt durch PSC + 1. CNT, Counter oder Zählwert, zählt aufwärts $0, \ldots,$ ARR; ARR ist das Auto-Reload Register. Jeder Zustand dauert einen ganzen Tick. Eine Periode hat $N = \text{ARR} + 1$ Intervalle. Beim Active-High-Ausgang gilt HIGH genau für CNT < CCR; CCR ist das Capture/Compare Register. Der Pin hat ideal die Pegel 0 V und 3,3 V. Andere Zählmodi und Polaritäten werden ausdrücklich angegeben. PWM bedeutet Pulsweitenmodulation, WDT Watchdog Timer.

Geben Sie Rechenweg, Einheit und die Bedeutung des Ergebnisses an. Alle Werte sind Modellannahmen, keine Messungen oder universellen Registervorgaben.

Aufgabe 1: Takt, Intervalle und Endpunkte (6 Punkte)

1. Für `PSC = 99`, `ARR = 999`, `CCR = 250`: Berechnen Sie Zählfrequenz, PWM-Frequenz, HIGH-Zeit und aktiven Anteil. Markieren Sie HIGH und LOW in einer Periode mit Zeitachse. (2 P)
2. Jemand ersetzt ARR durch `1000`, lässt PSC und CCR jedoch unverändert. Bleiben Frequenz und Tastverhältnis genau gleich? Geben Sie die neuen Werte und die Ursache der Abweichung an. (2 P)
3. Ein separates Auf/ab-Modell mit `ARR = 4` durchläuft $0, 1, 2, 3, 4, 3, 2, 1$ und danach wieder 0. Vergleichen Sie seine Periodendauer bei 1 MHz Zähltakt mit dem Aufwärtszählen $0, 1, 2, 3, 4$. Darf hier überall $N = \text{ARR} + 1$ stehen? (2 P)

Ihre Rechnung und Ihr Zeitdiagramm:

Aufgabe 2: Mittelwert, Effektivwert und Polarität (6 Punkte)

Eine ideale Spannungsquelle liefert das ungefilterte PWM-Rechteck 0/3,3 V mit HIGH-Anteil $D_H = 25\,\%$.

1. Bestimmen Sie den zeitlichen Mittelwert $\overline{V}$ und den Effektivwert V_{RMS}. RMS bedeutet Root Mean Square. (2 P)
2. Berechnen Sie die mittlere Leistung an $R = 1\ \mathrm{k\Omega}$. Welche Leistung hätte eine tatsächlich konstante Spannung $\overline{V}$ an demselben Widerstand? Erklären Sie den Unterschied. (2 P)
3. Nun sei der Ausgang Active-Low bei unverändertem aktiven Anteil von 25 %. Bestimmen Sie HIGH-Anteil und Spannungsmittelwert. Begründen Sie, weshalb aktive Lastzeit und HIGH-Anteil getrennt werden müssen. (2 P)

Ihre Rechnung und Interpretation:

Aufgabe 3: Flanken und gepufferte Compare-Werte (5 Punkte)

1. Software-PWM plant 50 μs HIGH, danach 50 μs LOW. Die fallende Flanke verspätet sich um 20 μs. Die anschließende LOW-Wartezeit bleibt 50 μs. Berechnen Sie neue Periode, Frequenz und HIGH-Anteil. (2 P)
2. Bei einem einzelnen Puls verschieben sich beide Flanken gleich um 5 μs. Ändert sich seine Breite? Was ändert sich trotzdem? (1 P)
3. Im Timer aus Aufgabe 1, Teil 1 wird CCR bei CNT 500 von 250 auf 750 geändert. Der ungepufferte Vergleich prüft kontinuierlich CNT $<$ CCR. Zeichnen Sie die HIGH-Abschnitte der betroffenen Periode und berechnen Sie ihren gesamten HIGH-Anteil. Vergleichen Sie mit einer gepufferten Übernahme erst beim nächsten Periodenanfang. (2 P)

Ihre Zeitdiagramme und Begründung:

Aufgabe 4: Servoformat und C-Rechenmodelle (5 Punkte)

Ein gewähltes Servoformat wiederholt Pulse alle 20 ms; 1,5 ms ist ein mögliches Kommando. Der Timer zählt mit 1 MHz.

1. Bestimmen Sie ARR, CCR und aktiven Anteil für diesen Puls. Wie lang wäre ein Puls mit demselben Anteil bei 1 kHz PWM? (2 P)
2. Die folgende reine Rechenfunktion rundet auf den nächsten Tick, bei genau halbem Tick nach oben. Welchen Wert liefert `percent_to_ccr(1, 49)`, und welchen tatsächlichen Anteil bedeutet er? Was liefert `percent_to_ccr(255, 999)`? (1 P)

```
uint32_t percent_to_ccr(uint32_t percent, uint16_t arr) {
    if (percent > 100u) percent = 100u;
    uint32_t n = (uint32_t)arr + 1u;
    return (percent * n + 50u) / 100u;
}
```

3. Eine Fading-Folge verwendet einen neuen Schritt alle 10 ms:

```
uint8_t fade_percent(uint64_t step) {
    uint32_t phase = (uint32_t)(step % 200u);
    return (uint8_t)(phase <= 100u ? phase : 200u - phase);
}
```

Welcher Wert entsteht bei `step = 125`? Wie lang ist ein voller Zyklus? Wie sollte eine verspätete Hauptschleife den aktuellen Schritt bestimmen, wenn alte Zwischenwerte übersprungen werden dürfen? Ändert eine verspätete Sollwertaktualisierung zwangsläufig die Hardware-PWM-Periode? (2 P)

Die Typen stammen aus `<stdint.h>`; `u` kennzeichnet unsigned-Konstanten. Beide Funktionen berechnen Werte ohne Gerätezugriff oder Wartefunktion.

Ihre Rechnung und Codeanalyse:

Aufgabe 5: Watchdogfristen und Zeitfenster (5 Punkte)

1. Ein idealer Watchdog hat zwei Sekunden Timeout. Feeds erfolgen bei 0, 0,5 und 1 s; danach fehlen sie. Das Hauptprogramm hängt ab 1,2 s. Wann ist der Reset, und wie lang kann das letzte PWM-Muster bis dahin noch weiterlaufen, falls Timer und Takt aktiv bleiben? (2 P)
2. Für denselben fest eingestellten Zähler sei das nominale Timeout zwei Sekunden, der Watchdogtakt aber um bis zu 20 % schneller oder langsamer. Berechnen Sie frühestes und spätestes Timeout. Passt ein größter vorgesehener Feedabstand von 1,4 s mit 0,2 s zusätzlicher Verzögerung vor das früheste Timeout? (2 P)
3. Ein separates Fenster-Modell erlaubt Feeds nur für $0{,}5\,\mathrm{s} < a < 2\,\mathrm{s}$; a ist das Alter seit dem letzten Feed. Beurteilen Sie $a = 0{,}2$ s, 1 s und 2,1 s. Ist bei $a = 1$ s ein Feed ohne frischen Anwendungsfortschritt sinnvoll? (1 P)

Ihre Zeitachse und Auslegung:

Aufgabe 6: Fortschritt, Reset und Aktorzustand (5 Punkte)

Eine Überwachungsrunde verlangt **neuen** Fortschritt bei Sensorerfassung, Regelberechnung und Ausgabe. Nach jedem erfolgreichen Feed werden die Nachweise für die nächste Runde zurückgesetzt. Die hier betrachteten Feedzeiten liegen im gültigen Fenster.

1. Ergänzen Sie für jede Runde „Feed ja/nein“ und begründen Sie. Ein altes gesetztes Flag zählt nicht als neue Aktivität. (2 P)

Runde	Neuer Sensorwert	Neue Berechnung	Neue Ausgabe	Feed?
A	ja	ja	ja	
B	ja	nein	ja	
C	altes Flag	altes Flag	altes Flag	

2. Eine Timer-ISR läuft weiter, während die Regelberechnung hängt. Weshalb würde blindes Füttern in dieser ISR den Fehler verdecken? Genügt allein das Verschieben des Feeds in die Hauptschleife? (1 P)
3. Beim Timeout startet die CPU neu. Nennen Sie zwei zu prüfende Randbedingungen, bevor daraus ein sicherer Motorzustand gefolgert werden darf. Erklären Sie außerdem, warum nach dem Neustart die Resetursache ausgewertet werden sollte. (2 P)

ISR bedeutet Interrupt Service Routine, CPU Central Processing Unit. Für die Resetdomänen des konkreten Geräts ist hier kein Verhalten vorgegeben.

Ihre Entscheidungen und Begründungen:

Optional: Rechenfunktionen am Host prüfen

Übernehmen Sie die beiden Funktionen aus Aufgabe 4 und die Funktion `may_feed` aus Vorlesung 4 in ein gewöhnliches C-Programm. Für `may_feed` benötigen Sie zusätzlich `<stdbool.h>`. Prüfen Sie beispielsweise:

- Prozentumsetzung: `0`, `25`, `100` und `255` bei ARR 999; außerdem `1` bei ARR 49 und `100` bei ARR 65535.
- Fading: Schritte 0, 100, 125, 199, 200; alle Ergebnisse sollen zwischen 0 und 100 liegen.
- Feedentscheidung: Alter 500, 501, 1999, 2000 ms, jeweils mit und ohne frischen Fortschritt.

Übersetzen Sie etwa mit `gcc -std=c11 -Wall -Wextra -Werror`. Die Hostausführung prüft Integerrechnung und Bedingungen, keine Pinflanken, Preload-Übernahme oder Resetdomänen. Ein Geräteversuch benötigt zuerst einen passenden dokumentierten Timer-/Watchdogtreiber sowie beobachtbare Ein- und Ausgänge; aus den Funktionsnamen entstehen keine Registeradressen.